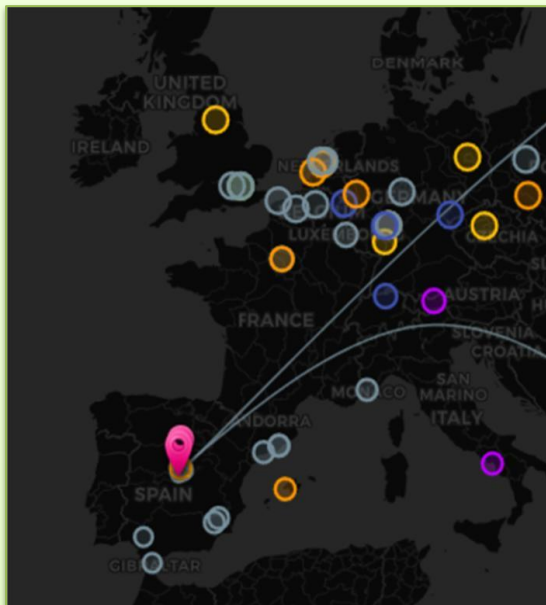




RED DE HONEYPOTS DISTRIBUIDA CON T-POT

Arquitectura Hive + Sensor sobre
infraestructura doméstica

IES Gaspar Melchor De Jovellanos

Grado Superior Técnico en Administración
de Sistemas Informáticos en Red

Profesor Coordinador:

Javier Ortiz Laguna

Autores:

Adrián Pérez Peláez

Izan Lozano Hernández

Víctor Manuel Humanes Ruiz

Promoción 2025-2026

ÍNDICE

Declaración de Originalidad y Autoría.....	2
Resumen.....	3
Abstract	4
1. Introducción y Marco Teórico.....	5
1.1. Justificación del proyecto	5
1.2. Objetivos	5
1.3. Significado del término.....	6
2. Metodología y Desarrollo	7
2.1. Arquitectura del sistema.....	7
2.2. Hardware y software utilizado.....	8
2.3. Decisiones técnicas justificadas.....	9
2.4. Planificación temporal	10
2.5. Seguridad perimetral	10
2.6. Desarrollo por semanas	11
Semana 1 — 23 al 29 de marzo: Planificación, infraestructura y seguridad perimetral Nodo 1.....	11
Semana 2 — 30 de marzo al 5 de abril: Instalación T-Pot Hive — Nodo 1.....	11
Semana 3 — 6 al 12 de abril: Preparación, instalación y seguridad perimetral Nodo 2.....	12
Semana 4 — 13 al 19 de abril: Instalación T-Pot Sensor e integración con Hive.....	12
Semana 5 — 20 al 26 de abril: Verificación del sistema y acceso remoto	12
Semana 6 — 27 de abril al 3 de mayo: Instalación de T-Pot Hive en el Nodo 1	13
3. Resultados y Discusión.....	13
3.1. Verificación del sistema.....	13
3.2. Análisis de ataques reales.....	14
3.3. Sistema de alertas Telegram.....	16
3.4. Comparativa con los objetivos iniciales.....	17
3.5. Problemas encontrados y soluciones.....	17
Ruta por defecto no persistente	17
Puerto 25 ocupado por exim4.....	17
Doble NAT por extensor de red	17
Certificado SSL de Logstash	18

4. Conclusiones.....	18
4.1. Cumplimiento de objetivos	18
4.2. Aprendizajes.....	18
4.3. Limitaciones.....	18
5. Referencias Bibliográficas.....	19
Anexos	20
Anexo A — Código fuente del bot de alertas Telegram	20
Anexo B — Capturas del dashboard de Kibana	22
Anexo C — Capturas del sistema de alertas Telegram	24
Anexo D — Capturas de la infraestructura desplegada	25

Declaración de Originalidad y Autoría

Nosotros los autores del Trabajo de Fin de Ciclo titulado “**Red de Honeypots Distribuida con T- Pot**”, presentado para la superación del módulo de Proyecto del ciclo formativo CFGS Administración de Sistemas Informáticos en Red en el IES Gaspar Melchor de Jovellanos, declaramos:

- Que el trabajo presentado es original y ha sido elaborado únicamente por nosotros.
- Que no se han utilizado fuentes, fragmentos de texto, código ni ideas de terceros sin su atribución en la bibliografía o en el documento.
- Que el uso de herramientas de ayuda en la redacción o el código ha sido complementario y supervisado en todo momento por nosotros.
- Que somos conscientes de que la detección de plagio o falta de originalidad puede suponer la anulación de la calificación del trabajo.

Fuenlabrada, mayo de 2026

Adrián Pérez Peláez
Hernández

Víctor Manuel Humanes Ruiz

Izan Lozano

Resumen

Este proyecto describe el diseño, despliegue y puesta en marcha de una **red de honeypots** distribuida basada en la plataforma T-Pot, sobre una infraestructura doméstica real. El sistema se compone de **dos nodos físicos** ubicados en distintas localizaciones geográficas (Madrid y Toledo), comunicados a través de una red privada virtual cifrada mediante **Tailscale**.

El Nodo 1 actúa como **servidor central** Hive, ejecutando la lista completa de T-Pot con Elasticsearch, Logstash y Kibana para recepcionar, unificar y visualizar eventos. El Nodo 2 opera en **modo Sensor**, capturando tráfico malicioso real proveniente de internet y reenviándolo al Hive a través del túnel cifrado. Ambos nodos reciben ataques de forma simultánea desde IPs públicas independientes, lo que permite una comparativa geográfica entre dos redes distintas.

El proyecto abarca desde la planificación de la arquitectura y el bastionado de ambos nodos, hasta la integración completa del sistema y la verificación del flujo de datos en tiempo real. Se incluye además un **sistema de alertas automáticas** por Telegram desarrollado en Python, que notifica con resúmenes de IPs atacantes y su país de origen. Se documentan los problemas encontrados durante el despliegue, especialmente los derivados del uso de routers domésticos con funciones limitadas, y sus soluciones en cada caso.

Palabras clave: honeypot, T-Pot, ciberseguridad, Hive, Sensor, Elasticsearch, Kibana, Tailscale, Docker, Debian, red distribuida.

Abstract

This project describes the design, deployment and commissioning of a distributed honeypot network based on the T-Pot platform, on a real domestic infrastructure. The system consists of two physical nodes located in different geographical locations, communicating through an encrypted virtual private network using Tailscale.

Node 1 acts as the central Hive server, running the full T-Pot stack with Elasticsearch, Logstash and Kibana for receiving, indexing and visualising events. Node 2 operates in Sensor mode, capturing real malicious traffic from the internet and forwarding it to the Hive through the encrypted tunnel. Both nodes receive attacks simultaneously from independent public IPs, enabling a real geographical comparison between two distinct networks.

The project covers everything from architecture planning and hardening of both nodes, to full system integration and real-time data flow verification. A Python-based Telegram alert system is also included, notifying the team with summaries of attacking IPs and their country of origin. Problems encountered during deployment are documented, especially those arising from the use of domestic routers with limited functionality, and the solutions applied in each case.

Keywords: honeypot, T-Pot, cybersecurity, Hive, Sensor, Elasticsearch, Kibana, Tailscale, Docker, Debian, distributed network.

1. Introducción y Marco Teórico

1.1. Justificación del proyecto

Las amenazas de ciberseguridad a día de hoy evolucionan de forma constante y los ataques automatizados contra sistemas conectados a internet son cada vez más. Cualquier equipo con una IP pública puede comenzar a recibir intentos de intrusión en cuestión de minutos desde su exposición. Este hecho, documentado en el ámbito profesional, nos motivó el planteamiento de este proyecto, que nos permitiera estudiar, capturar y analizar estos ataques de forma controlada y segura.

Este proyecto se enmarca directamente en los requerimientos del ciclo formativo CFGS ASIR, especialmente en los módulos de Seguridad y Alta Disponibilidad, Administración de Sistemas Operativos y Servicios de Red e Internet. La implementación práctica de una infraestructura sobre hardware doméstico nos permite desarrollar competencias que van más allá del entorno de laboratorio controlado.

1.2. Objetivos

El objetivo principal del proyecto es desplegar una red de honeypots distribuida y funcional sobre una infraestructura doméstica, capaz de capturar, registrar y analizar ataques de internet de forma simultánea desde dos ubicaciones geográficas independientes.

Los objetivos son:

1. **Instalar y configurar T-Pot** en modo Hive sobre el **Nodo 1**, desplegando la lista completa de análisis con Elasticsearch, Logstash y Kibana.
2. **Instalar y configurar T-Pot** en modo Sensor sobre el **Nodo 2**, enlazándolo al Hive mediante el script oficial de despliegue.
3. Aplicar **seguridad perimetral** en capas en ambos nodos: DMZ, port forwarding selectivo, UFW, Fail2ban y hardening del sistema operativo.
4. Establecer **acceso remoto** seguro para los tres miembros del equipo mediante **Tailscale** VPN.
5. Desarrollar un **sistema de alertas** automáticas por Telegram que nos notifique de actividad maliciosa en tiempo real.

6. **Documentar los problemas** encontrados durante el despliegue y las soluciones aplicadas.
7. Realizar **pruebas controladas** y verificar que el sistema registra y visualiza correctamente los ataques.

1.3. Significado del término

Un honeypot es un sistema diseñado a propósito para ser atacado. Su propósito no es proteger cosas reales sino atraer a los atacantes, registrar sus acciones y obtener información sobre las técnicas y métodos de ataque utilizados.

Los honeypots se clasifican según su nivel de interacción con el atacante. Los de baja interacción simulan servicios de forma superficial y registran conexiones y credenciales, siendo seguros pero limitados en la información que capturan. Los de alta interacción exponen sistemas reales o emulaciones difíciles o complejas, obteniendo más información, pero con mayor riesgo. T-Pot utiliza principalmente honeypots de baja interacción contenedorizados (docker), lo que permite ejecutar más de veinte en paralelo sin comprometer la seguridad del sistema que sea el anfitrión.

T-Pot CE (Community Edition), ha sido desarrollado y mantenido por Telekom Security, es actualmente una de las plataformas de honeypot más completas disponibles en código abierto. Integra honeypots especializados en distintos protocolos (SSH, Telnet, HTTP, RDP, SMB, FTP...), junto con una pila ELK completa para el análisis y visualización de eventos con herramientas complementarias como un mapa de ataques en tiempo real.

La arquitectura Hive + Sensor, permite desplegar un nodo central que centraliza el análisis y múltiples sensores remotos que solo capturan ataques. Esta arquitectura replica el modelo utilizado en entornos profesionales con sensores distribuidos en distintas sedes, y es la que nosotros implementamos sobre una infraestructura doméstica.

2. Metodología y Desarrollo

2.1. Arquitectura del sistema

La arquitectura del sistema se basa en el modelo **Hive + Sensor de T-Pot**, adaptado a nuestra infraestructura doméstica con dos nodos físicos en ubicaciones geográficas distintas. Ambos nodos están expuestos a internet desde sus propias IPs públicas y capturan ataques de forma simultánea, mientras que la visualización y el análisis se centralizan en el **Nodo 1**.

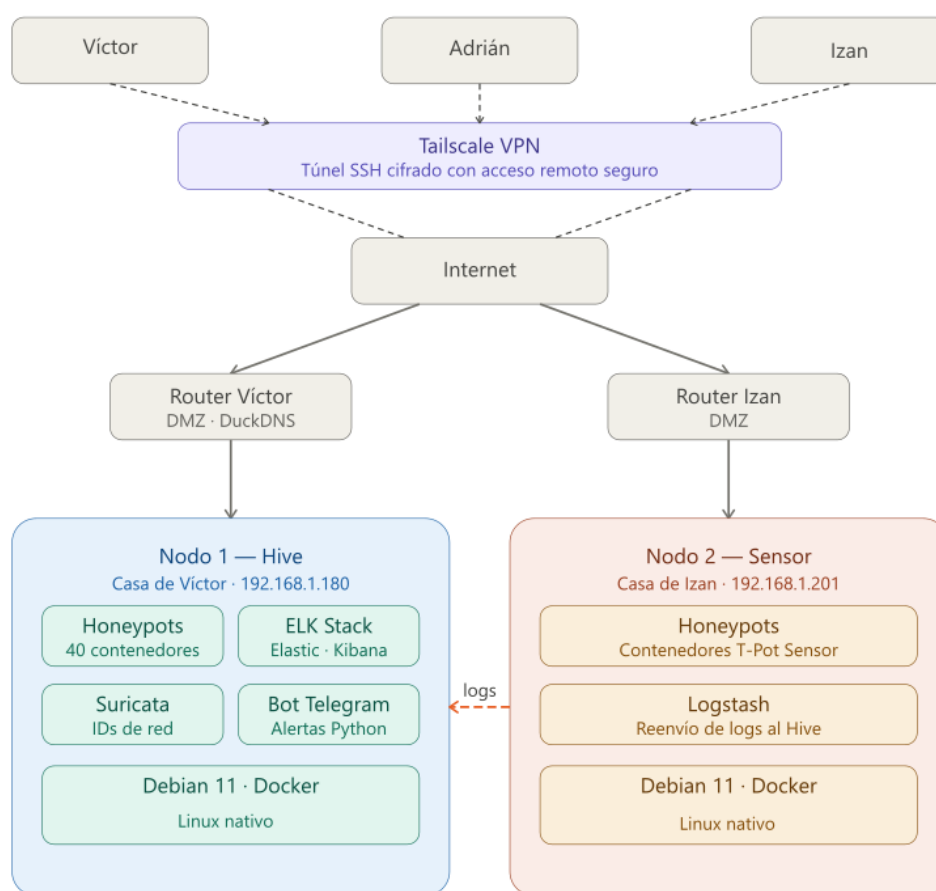


Diagrama de arquitectura Hive + Sensor con flujo de datos entre nodos

La comunicación entre nodos se establece mediante **un túnel SSH** que está cifrado sobre la **red de Tailscale**, garantizando que los logs lleguen al Hive y al sensor de forma segura sin exponer puertos de administración (los que utilizamos) a internet. Nuestro acceso remoto al servidor también se realiza exclusivamente a través de Tailscale.

La arquitectura esta formada por los siguientes puntos:

- **Administradores:** equipos de los administradores (nosotros), Adrián, Izan y Victor.
- **Tailscale:** Un túnel ssh cifrado por el cuál nos podemos conectar remotamente a los nodos pasando por internet.
- **Routers:** Ambos routers que se encuentran en nuestros domicilios, los cuales tienen sus propias DMZ donde están alojados los nodos.
- El **nodo 1 (Hive)**, el cuál esta en la casa de Victor, en el que tenemos instalado y configurado lo siguiente:
 - **Honeypots:** 40 contenedores (cada contenedor es 1 servicio vulnerable).
 - **ELK Stack:** (formado por elastic y kibana). Estos nos hacen posible ver visualmente los datos mediante gráficas, etc...
 - **Suricata:** (servicio que detecta ciertos tipos de tráfico en la red, generando logs en formato. json para posteriormente pasarlos al siem).
 - **Bot de telegram:** (Bot creado mediante un script que lee los logs y avisa mediante un grupo conformado por los administradores con avisos sobre intrusiones al honeypot).
 - **Debian 11 – Docker:** (Sistema Operativo utilizado con Linux nativo).
- El **nodo 2 (Sensor)**, el cuál esta en la casa de Izan, formado por:
 - **Contenedores T-pot Sensor:** (servicios vulnerables que funcionan como sensores enviando logs al nodo 1).
 - **Logstash:** (script que envía logs al nodo 1 siendo posible la comunicación entre ambos).
 - **Debian 11 – Docker:** (Sistema Operativo utilizado con Linux nativo).

2.2. Hardware y software utilizado

Componentes	Nodo 1 (Hive)	Nodo 2 (Sensor)
Representación:	Servidor central	Sensor remoto
RAM:	11 GB	8 GB
Almacenamiento:	3 discos HDD (137 GB SO, 313 GB /data, 916 GB /backup)	1 disco HDD particionado (512 MB /boot, 50GB /primaria, 4GB /swap, 96 GB /data)
Ubicación:	Casa de Víctor (Toledo)	Casa de Izan (Madrid)
Sistema operativo:	Debian 11 (Bullseye) sin escritorio	Debian 11 (Bullseye) sin escritorio
Conexión:	Cable, IP dinámica + DuckDNS	Cable, IP dinámica + DuckDNS
Router:	Router de ADAMO (operador)	Zyxel AX7501-B0 (Digi)

Software	Versión / Detalle
T-Pot CE	Repositorio oficial telekom-security/tpotce (la versión actualizada de 2025)
Debian	11 Bullseye, sin entorno gráfico
Docker	29.4.0
Docker Compose	v5.1.1
Elasticsearch	Incluido en T-Pot (pila ELK)
Tailscale	VPN mesh, plan gratuito
DuckDNS	DNS dinámico gratuito
UFW + Fail2ban	Firewall y protección de fuerza bruta
Python 3	Script de alertas Telegram

2.3. Decisiones técnicas justificadas

A lo largo del proyecto hemos tomado varias decisiones técnicas relevantes que han condicionado el resultado final:

- **Migración del repositorio:** el proyecto partía de un fork privado de T-Pot (versión 20.06, 2020) con más de 600 commits de retraso respecto al oficial y problemas de compatibilidad con Docker moderno. Lo migramos al repositorio oficial de Telekom Security para garantizar estabilidad y acceso a la arquitectura Hive + Sensor completa.
- **Sustitución del hardware:** el PC inicialmente pensado para el Nodo 1 (Xeon X5550, 8 GB RAM) fue descartado por no dejar margen operativo sobre el mínimo requerido por T- Pot. Lo sustituimos por un servidor con 11 GB de RAM.
- **Arquitectura de dos víctimas:** elegimos la modalidad en que ambos nodos reciben ataques simultáneamente (en lugar de que solo el Sensor sea víctima) para obtener una comparativa geográfica real entre las dos IPs públicas.
- **Desactivación de SSL en Logstash:** el certificado auto firmado del Hive no incluía la IP de Tailscale como nombre alternativo, asique se rechazaba por parte de Logstash en el Sensor. Lo resolvimos desactivando la verificación SSL mediante la variable `LS_SSL_VERIFICATION=none`.

2.4. Planificación temporal

Semana	Periodo	Contenido
Semana 1	23–30 mar	Planificación, selección de hardware y repositorio, diseño de arquitectura, anteproyecto
Semana 2	31 mar–6 abr	Instalación Debian 11 Nodo 1, T-Pot Hive, bastionado
Semana 3	7–13 abr	Preparación e instalación Debian 11 Nodo 2, bastionado
Semana 4	14–20 abr	Instalación T-Pot Sensor, integración con Hive, deploy.sh
Semana 5	8 abr	Verificación acceso remoto, Tailscale, Docker, comprobación discos y RAM
Semana 6	8 abr	Instalación definitiva T-Pot Hive Nodo 1, 40 contenedores, dashboard
Semana 7	May 2026	Bot de Telegram, pruebas controladas, captura de ataques reales
Semana 8	May 2026	Análisis de resultados, redacción de memoria

2.5. Seguridad perimetral

La seguridad de las redes domésticas donde están los nodos ha sido la prioridad del proyecto. Implementamos una arquitectura de seguridad en capas:

- **Capa 1 — Router:** DMZ con port forwarding selectivo, exponiendo únicamente los puertos definidos por los honeypots activos de T-Pot. Los puertos de administración (64294, 64295, 64297) nunca se exponen a internet.
- **Capa 2 — Firewall UFW:** configurado en cada servidor para bloquear todo tráfico no autorizado a nivel de host, como segunda capa independiente del router.
- **Capa 3 — Fail2ban:** protección contra fuerza bruta en los puertos de administración accesibles por Tailscale.

- **Capa 4 — Tailscale VPN:** todo el acceso administrativo (SSH, Kibana, Cockpit) se realiza exclusivamente a través de la VPN mesh privada.
- **Capa 5 — Hardening del SO:** puerto SSH cambiado al no estándar, acceso root por SSH desactivado, actualizaciones de seguridad automáticas activadas.

2.6. Desarrollo por semanas

Semana 1 — 23 al 29 de marzo: Planificación, infraestructura y seguridad perimetral Nodo 1

Durante esta primera semana, resolvimos las decisiones de base que condicionaron el resto del proyecto. El primer punto fue el hardware: el PC inicialmente previsto para el servidor (Intel Xeon X5550, 8 GB de RAM) fue descartado al comprobarse que 8 GB era el mínimo estricto requerido por T-Pot y no dejaba margen suficiente. Se sustituyó por un segundo servidor de torre con 11 GB de RAM, que pasó a ser el Nodo 1 definitivo y proporcionó el margen necesario para ejecutar todos los contenedores de T-Pot con estabilidad.

El segundo punto fue la elección del repositorio. El proyecto partía de un fork privado de T-Pot ([github-rashelbach/-T-Pot-Honeypot](https://github.com/rashelbach/T-Pot-Honeypot), basado en la versión 20.06 de 2020), pero tras analizarlo se detectó que llevaba más de 600 commits de retraso respecto al repositorio oficial y presentaba problemas de compatibilidad con versiones modernas de Docker. Se tomó la decisión técnica justificada de migrar al repositorio oficial de Telekom Security ([telekom-security/tpotce](https://github.com/telekom-security/tpotce)).

El tercer punto fue la arquitectura de captura. Se valoraron tres opciones: un único nodo, un Hive con Sensor dedicado exclusivamente a recibir ataques, y una arquitectura con dos nodos que recibieran ataques de forma simultánea desde IPs públicas independientes. Se optó por esta última para obtener comparativa geográfica real. Se elaboró el anteproyecto y se entregó al tutor.

Se verificó que Tailscale estaba operativo en el Nodo 1 y se realizó una prueba de conectividad remota desde los tres equipos del equipo. A continuación, se instalaron las dependencias base y se procedió al bastionado: DMZ con port forwarding selectivo, UFW, Fail2ban, cambio de puerto SSH, desactivación de root y configuración de DuckDNS.

Semana 2 — 30 de marzo al 5 de abril: Instalación T-Pot Hive — Nodo 1

La semana comenzó con la instalación de Debian 11 (Bullseye) en el Nodo 1, utilizando la imagen netinst oficial. Se realizó el particionado manual de los tres discos duros: el de 137 GB para el sistema operativo y Docker, el de 313 GB con punto de montaje /data para logs y datos de T-Pot, y el de 916 GB con punto de montaje /backup para copias de seguridad.

Se procedió al despliegue de T-Pot en modo Hive. Se clonó el repositorio oficial y se ejecutó el instalador, que configuró automáticamente todos los contenedores Docker. Se optimizó Elasticsearch ajustando los parámetros de memoria heap y se estableció una política de retención de 30 días.

Semana 3 — 6 al 12 de abril: Preparación, instalación y seguridad perimetral Nodo 2

Esta semana se centró en la preparación del Nodo 2, ubicado en casa de Izan. Se realizó una instalación limpia de Debian 11 con particionado manual: /boot primaria, swap lógica de 4 GB, / primaria de 50 GB y /data con el espacio restante. Se instalaron las dependencias base, se autenticó el equipo en Tailscale y se aplicó el mismo esquema de bastionado que en el Nodo 1.

El router del Nodo 2 es un Zyxel AX7501-B0 de Digi que requiere el servicio Conexión Plus para la apertura de puertos. Adicionalmente, el servidor estaba conectado a través de un extensor de red en modo repetidor, lo que generaba doble NAT y requirió ajustes adicionales. Se configuró DuckDNS con el subdominio nodo2honeypot.

Semana 4 — 13 al 19 de abril: Instalación T-Pot Sensor e integración con Hive

Esta semana fue la más importante del proyecto, ya que supuso la integración completa de los dos nodos. Se instaló T-Pot en modo Sensor sobre el Nodo 2. Durante el primer arranque se detectó que el servicio exim4, instalado por defecto en Debian, ocupaba el **puerto 25** necesario para T-Pot, por lo que se desactivó el servicio y los contenedores arrancaron correctamente.

Para enlazar el Sensor con el Hive ejecutamos el script oficial deploy.sh desde el Nodo 1, apuntando a la IP de Tailscale del Nodo 2. Logstash del Nodo 2 no podía conectarse al Hive por un error de validación del certificado SSL auto firmado. Lo que hicimos fue desactivar la verificación SSL en Logstash mediante la variable **LS_SSL_VERIFICATION=none**. Así, los datos del Nodo 2 comenzaron a aparecer en Kibana del Nodo 1, con **1.500 eventos capturados**.

Semana 5 — 20 al 26 de abril: Verificación del sistema y acceso remoto

El primer acceso remoto al servidor desde el portátil de Adrián reveló un problema con el puerto SSH. Al intentar conectarse al puerto 22 se obtuvo conexión rechazada (capturado por Cowrie); el puerto 64295 también falló porque T-Pot aún no estaba instalado. La conexión fue posible finalmente a través del puerto 2222, configurado por Víctor durante la puesta a punto inicial.

Se verificó Tailscale (active running, IP 100.92.112.74), los tres dispositivos en el panel (etokeh- book-pro 100.121.179.57, server 100.92.112.74, tuvieja 100.64.153.108), los discos montados (137 GB, data 313 GB, backup 916 GB), la zona horaria Europe/Madrid y la RAM disponible de 11 GB. Se resolvió el problema de compatibilidad del terminal xterm-ghostty forzando TERM=xterm-256color. Se instaló Docker 29.4.0 y Docker Compose v5.1.1.

Semana 6 — 27 de abril al 3 de mayo: Instalación de T-Pot Hive en el Nodo 1

Se clonó el repositorio oficial y se ejecutó el instalador. La instalación encontró un conflicto GPG entre docker.asc y docker.gpg (Conflicting values set for option Signed-By), resuelto eliminando los ficheros en conflicto y relanzando el instalador. T-Pot desplegó correctamente 40 contenedores Docker. Durante la instalación el puerto SSH fue reasignado automáticamente al 64295.

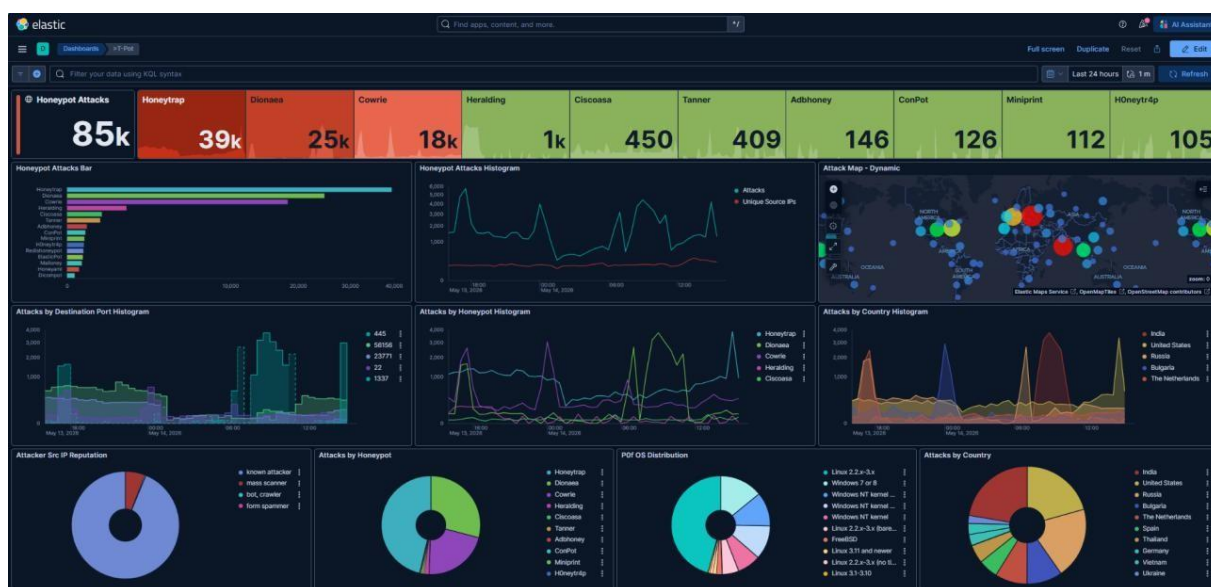
Al acceder al dashboard por primera vez el navegador mostró NET::ERR_CERT_AUTHORITY_INVALID por el certificado SSL auto firmado de T-Pot, comportamiento esperado. Se accedió aceptando la excepción de seguridad.

3. Resultados y Discusión

3.1. Verificación del sistema

Antes de proceder al análisis de ataques reales, se realizó una prueba controlada simulando ataques SSH, HTTP y otros protocolos desde equipos propios del equipo contra ambos nodos. Esta prueba permitió verificar que:

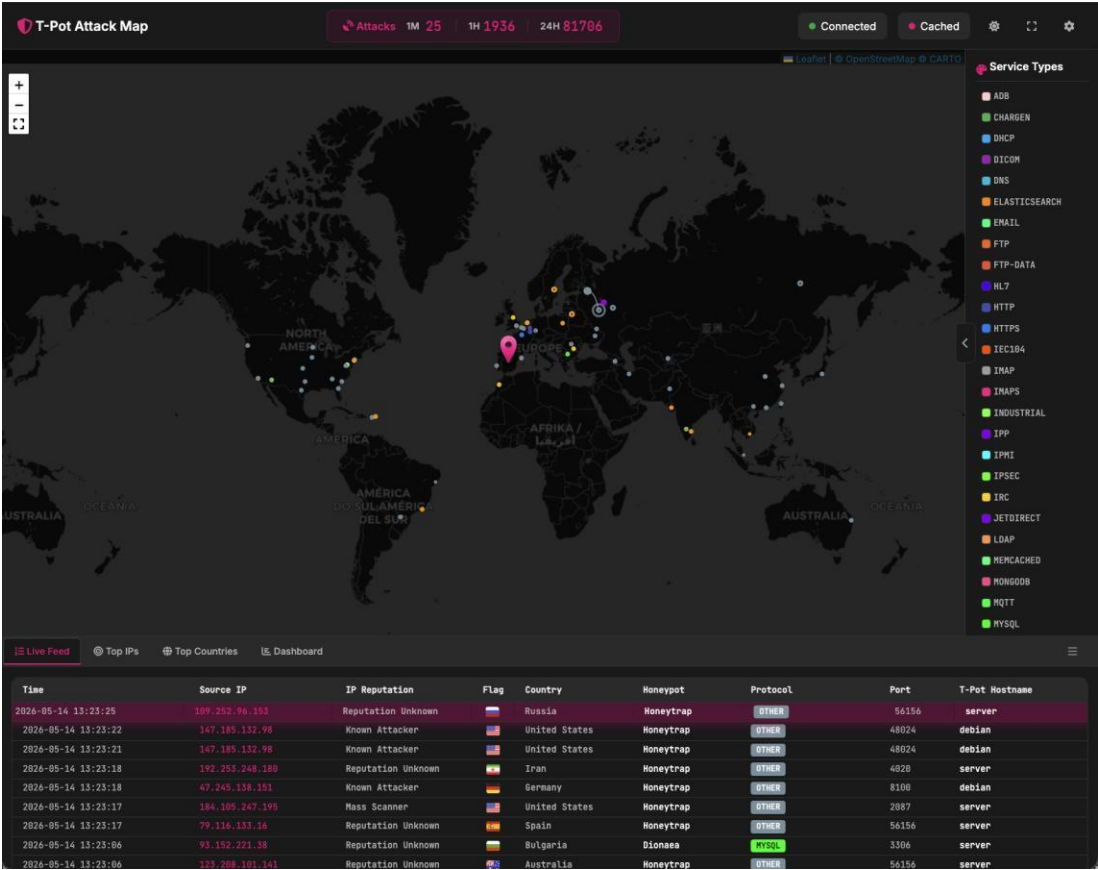
- Los **contenedores honeypot** de T-Pot responden correctamente en todos los puertos definidos.
- Los eventos generados aparecen correctamente indexados en **Elasticsearch** y visibles en **Kibana**.
- El **flujo de datos** del Nodo 2 al Nodo 1 funciona de forma continua y sin pérdida de eventos.
- El **bot de Telegram** recibe y envía correctamente los resúmenes de actividad al grupo del equipo.



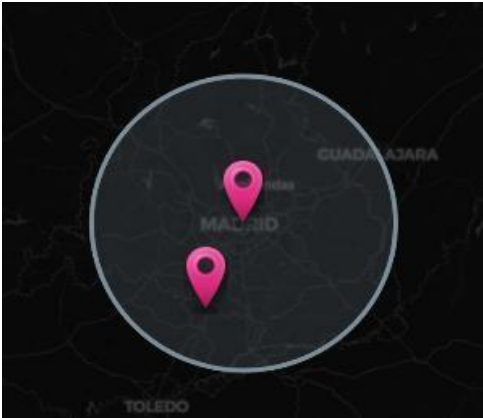
Dashboard de Kibana con su vista general de eventos

3.2. Análisis de ataques reales

A continuación, se presentan los resultados obtenidos durante el período de exposición a internet de ambos nodos. Los datos recogidos incluyen el volumen de eventos, los países de origen más activos, los puertos y servicios más atacados, y una comparativa entre los dos nodos.



Mapa de ataques en tiempo real y procedencia geográfica de los atacantes



Los dos nodos operativos visibles en el mapa.

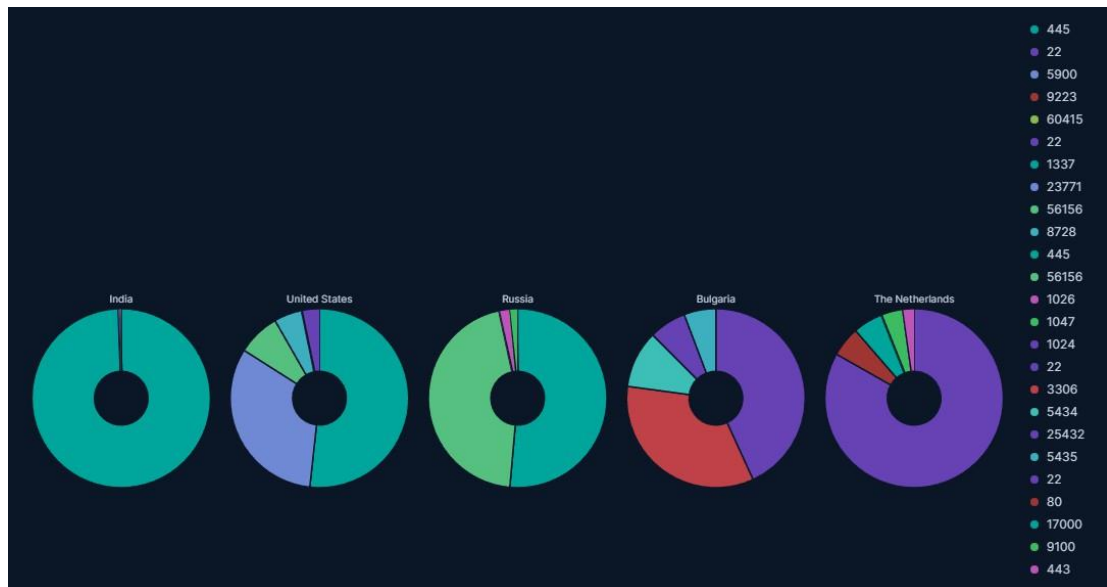


Gráfico de Kibana: top puertos atacados por protocolo

Rank	Hits	Source IP	Source IP Reputation	Flag	Country	Last Protocol Seen	Last Seen
1	39	157.238.138.141	Reputation Unknown		United States	OTHER	2026-05-14 13:26:10
2	30	79.116.133.16	Reputation Unknown		Spain	OTHER	2026-05-14 13:30:25
3	30	31.131.184.68	Reputation Unknown		Ukraine	OTHER	2026-05-14 13:30:16
4	30	172.58.8.42	Reputation Unknown		United States	OTHER	2026-05-14 13:30:37
5	30	176.115.149.160	Reputation Unknown		Russia	OTHER	2026-05-14 13:30:34
6	29	24.139.131.48	Reputation Unknown		Puerto Rico	OTHER	2026-05-14 13:29:55
7	29	93.176.157.129	Reputation Unknown		Spain	OTHER	2026-05-14 13:29:57
8	29	184.54.46.40	Reputation Unknown		United States	OTHER	2026-05-14 13:30:15
9	29	109.252.96.153	Reputation Unknown		Russia	OTHER	2026-05-14 13:30:38

Tabla de top IPs atacantes

3.3. Sistema de alertas Telegram

Se implementó un bot de Telegram en Python que monitoriza en tiempo real las IPs que generan tráfico malicioso detectado por UFW en los logs del kernel (/var/log/kern.log). El bot geolocaliza cada IP nueva mediante la API de ipinfo.io y envía resúmenes periódicos al grupo compartido del equipo, configurado para recibir notificaciones cada 40 IPs nuevas o cada 6 minutos (3600 segundos), lo que se produce primero.

Cada resumen incluye la hora del informe, el número de IPs nuevas detectadas en el periodo y el top 5 de países atacantes con su recuento. El bot se ejecuta como servicio systemd (honeypot-alertas) para garantizar su arranque automático y persistencia ante reinicios del servidor.

Grupo de Telegram



Resumen de alertas recibido por el equipo



3.4. Comparativa con los objetivos iniciales

A continuación, se evalúa el grado de cumplimiento de cada objetivo específico planteado al inicio del proyecto:

Objetivo	Estado	Observaciones
T-Pot Hive en Nodo 1	Completado	40 contenedores activos, Kibana operativo
T-Pot Sensor en Nodo 2	Completado	Integrado con Hive vía deploy.sh
Seguridad perimetral ambos nodos	Completado	5 capas aplicadas en cada nodo
Acceso remoto Tailscale	Completado	Los 3 miembros conectados
Bot de alertas Telegram	Completado	Servicio systemd activo
Documentación de problemas	Completado	5 problemas documentados con solución
Pruebas controladas	Completado	Verificado con ataques propios

3.5 Problemas encontrados.

Soluciones DNS no persistente

El servidor de DNS cambiaba cada reinicio. Solución:

Sudo nano /etc/resolv.conf

Buscar **nameserver** y cambiarlo a **8.8.8.8**

Ruta por defecto no persistente

La ruta por defecto desaparecía tras cada reinicio. Solución:

sudo ip route add default via 192.168.1.1 dev enp5s0f0

Puerto 25 ocupado por exim4

El servidor de correo exim4, instalado por defecto en Debian, ocupaba el puerto 25 necesario para T-Pot. Solución:

sudo systemctl stop exim4 && sudo systemctl disable exim4

Doble NAT por extensor de red

El Nodo 2 estaba conectado a través de un extensor Wifi en modo repetidor, generando doble NAT que impedía el correcto funcionamiento de la DMZ. El modo punto de acceso requiere cable al router, no es viable en la distribución física de la vivienda. Se resolvió configurando manualmente la ruta de red y el DNS en el servidor.

Certificado SSL de Logstash

El certificado SSL auto firmado del Nodo 1 no incluía la IP de Tailscale como nombre alternativo, por lo que Logstash del Nodo 2 lo rechazaba. Los cambios al importar el certificado se perdían al reiniciar los contenedores. Solución definitiva: desactivar la verificación SSL en Logstash mediante la variable de entorno `LS_SSL_VERIFICATION=none`.

4. Conclusiones

4.1. Cumplimiento de objetivos

El proyecto ha alcanzado su objetivo: desplegar una red de honeypots distribuida y funcional sobre infraestructura doméstica, con ambos nodos capturando ataques de forma simultánea y los datos centralizados en un único dashboard de Kibana. La arquitectura Hive + Sensor ha demostrado ser robusta y escalable, permitiendo añadir nuevos sensores con facilidad.

4.2. Aprendizajes

Uno de los aprendizajes más completos ha sido la gestión de las limitaciones sobre el uso de una infraestructura doméstica. Los routers de operadores con funcionalidades reducidas, las IPs dinámicas, el doble NAT generado por extensores de red o la presencia de servicios del sistema en conflicto con los puertos necesarios son problemas reales que no aparecen en entornos de laboratorio controlados.

El proyecto ha permitido adquirir competencias prácticas en: administración de sistemas Linux, configuración de redes y seguridad perimetral, despliegue y gestión de contenedores Docker, funcionamiento de pipelines de datos con Elasticsearch y Logstash, desarrollo de scripts Python para automatización, y análisis de tráfico malicioso real.

4.3. Limitaciones

Las principales limitaciones encontradas durante el proyecto fueron:

- Los problemas de persistencia de DNS y ruta de red en el Nodo 2 lo resolvimos de forma permanente.
- El certificado SSL auto firmado no fue regenerado con la IP de Tailscale incluida, lo que obligó a desactivar la verificación SSL en Logstash en lugar de resolver el problema en origen.
- La ausencia de un tercer nodo sensor nos limitó la comparativa geográfica a dos ubicaciones.

5. Referencias Bibliográficas

- **Telekom Security. T-Pot CE — Community Edition:** Repositorio oficial en GitHub: <https://github.com/telekom-security/tpotce>
- **Documentación oficial de Elasticsearch, Logstash y Kibana (ELK Stack):** <https://www.elastic.co/docs>
- **Documentación oficial de Tailscale VPN:**
<https://tailscale.com/kb>
- **Documentación del servicio de DNS dinámico (DuckDNS):**
<https://www.duckdns.org/install.jsp>
- **Manual de instalación de Debian 11 Bullseye**
<https://www.debian.org/releases/bullseye/installmanual>
- **UFW — Uncomplicated Firewall.**
Documentación de Ubuntu:
<https://help.ubuntu.com/community/UFW>
- **Fail2ban.** Documentación oficial:
https://www.fail2ban.org/wiki/index.php/Main_Page
- **Documentación oficial de Docker y Docker Compose:**
<https://docs.docker.com>
- **Documentación de la API de geolocalización de IPs (ipinfo.io):** <https://ipinfo.io/developers>

6. Anexos

Anexo A — Código fuente del bot de alertas Telegram

A continuación, incluimos el código fuente completo del script Python desarrollado para el sistema de alertas automáticas. El script monitoriza los logs del kernel en busca de nuevas IPs externas, las geolocaliza mediante la API de ipinfo.io y envía resúmenes periódicos a nuestro grupo de Telegram. El umbral de envío está configurado en 40 IPs nuevas o 6 minutos (3600 segundos), lo que se produzca primero, y puede modificarse ajustando las variables **LIMITE_ALERTAS** e **INTERVALO_MAXIMO** al inicio del fichero.

```
#!/usr/bin/env python3
import re
import ipinfo
import requests
import time
from datetime import datetime
from collections import Counter

TELEGRAM_TOKEN = "8768647124:AAE-
YWmmj3VyGwgnwZ3Gt88oOSwnkDgvkcM"
CHAT_ID = "-5095628985"
IPINFO_TOKEN = "2b914523aa4128"
LIMITE_ALERTAS = 40
INTERVALO_MAXIMO = 3600
LOG_FILE = "/home/via/tpotce/data/honeytrap/log/honeytrap.log"

handler = ipinfo.getHandler(IPINFO_TOKEN)

def enviar(texto):
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
    requests.post(url, json={"chat_id": CHAT_ID, "text": texto, "parse_mode":
"HTML"}, timeout=10)

def get_pais(ip):
    try:
        details = handler.getDetails(ip)
        return getattr(details, 'country_name', 'Desconocido')
    except:
        return 'Desconocido'

def enviar_resumen(nuevas):
    paises = Counter()
    for ip in nuevas:
        paises[get_pais(ip)] += 1
    top5 = paises.most_common(5)
    top_str = "\n".join([f"👉 {p}: {c} ataques" for p, c in top5])
    mensaje = (
        f"🚩<b>Resumen de ataques — LocalHoneyPot</b>\n\n"
        f"🕒 Hora: {datetime.now().strftime('%d/%m/%Y %H:%M:%S')}\n"
```



```

        f"    IPs nuevas detectadas: {len(nuevas)}\n\n"
        f"🚩 Top 5 países atacantes:\n{top_str}"
    )
    enviar(mensaje)
    print(f"Resumen enviado: {len(nuevas)} IPs")

    🚩<b>LocalHoneypot activo</b>\nBot iniciado correctamente.")

# Posicionarse al final del fichero
with open(LOG_FILE, 'r') as f:
    f.seek(0, 2)
    posicion = f.tell()

print(f"Monitorizando desde el final del log...")
nuevas_acumuladas = set()
ultimo_envio = time.time()

while True:
    time.sleep(60)
    with open(LOG_FILE, 'r') as f:
        f.seek(posicion)
        lineas_nuevas = f.read()
        posicion = f.tell()

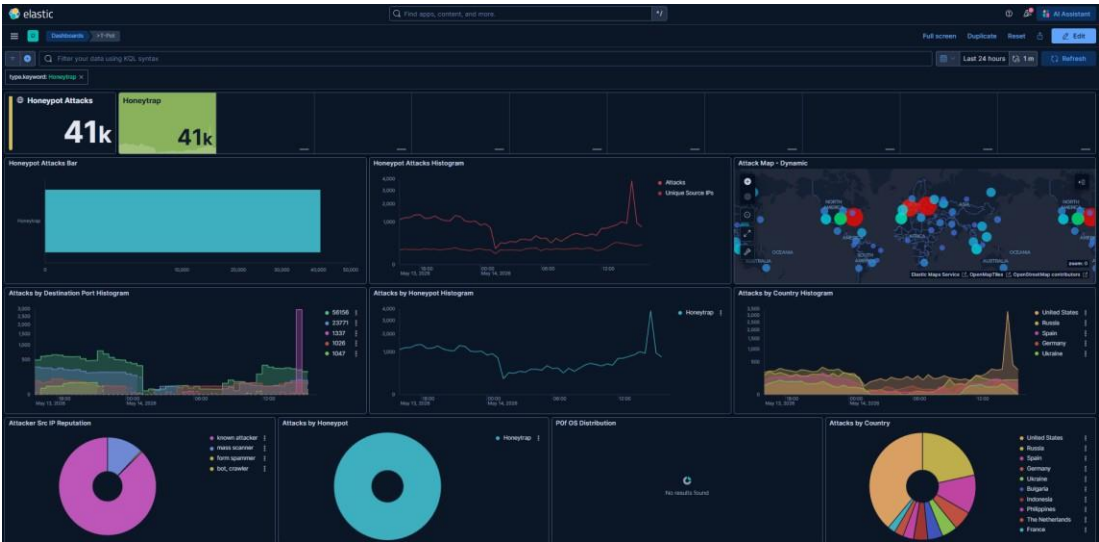
    ips = re.findall(r'(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}):\d+', lineas_nuevas)
    nuevas = {ip for ip in ips if ip and not ip.startswith('192.168') and not
ip.startswith('10.') and not ip.startswith('100.')}
    nuevas_acumuladas.update(nuevas)
    print(f"IPs nuevas este minuto: {len(nuevas)} | Acumuladas:
{len(nuevas_acumuladas)}")

    tiempo = time.time() - ultimo_envio
    if len(nuevas_acumuladas) >= LIMITE_ALERTAS or tiempo >=
INTERVALO_MAXIMO:
        if nuevas_acumuladas:
            enviar_resumen(nuevas_acumuladas)
            nuevas_acumuladas = set()
            ultimo_envio = time.time()

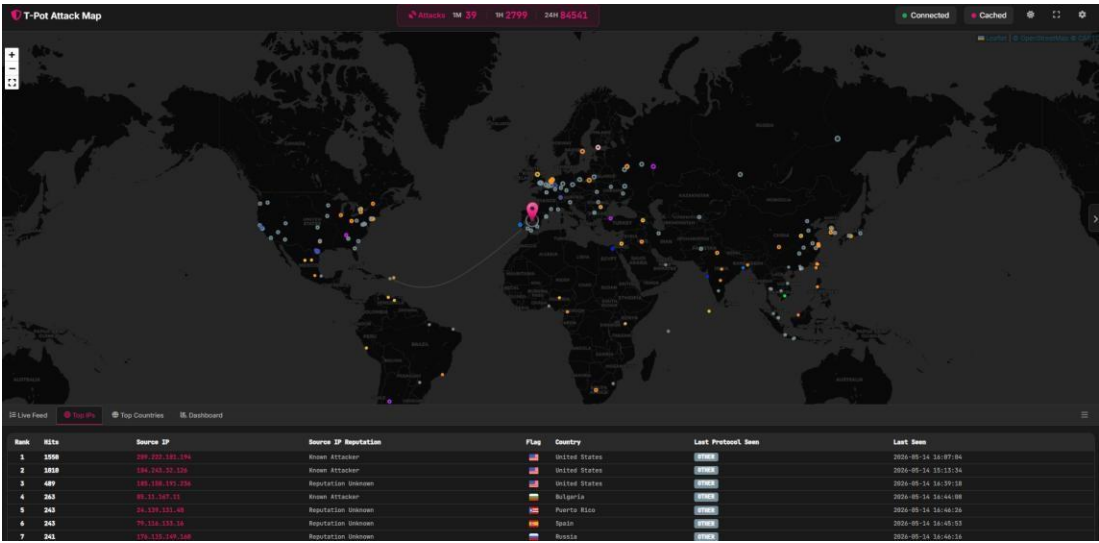
```

Anexo B — Capturas del dashboard de Kibana

En este anexo se recogen las principales capturas del panel de Kibana obtenidas durante el período de funcionamiento del sistema.



Kibana: detalle de eventos por honeypot



Kibana: distribución geográfica de ataques en mapa



Kibana: top credenciales utilizadas en intentos SSH



Usuario utilizado

Kibana: ataque simulado de Izan Lozano al Honeypot



Intentos de contraseñas utilizados

Kibana: ataque simulado de Izan Lozano al Honeypot

```

izani@izqnl:/mnt/c/Users/izani$ ssh root@143.131.214.116
Password:
Password:
Password:
root@143.131.214.116's password:
Permission denied, please try again.
root@143.131.214.116's password:
Permission denied, please try again.
root@143.131.214.116's password:
root@143.131.214.116: Permission denied (publickey,password,keyboard-interactive).

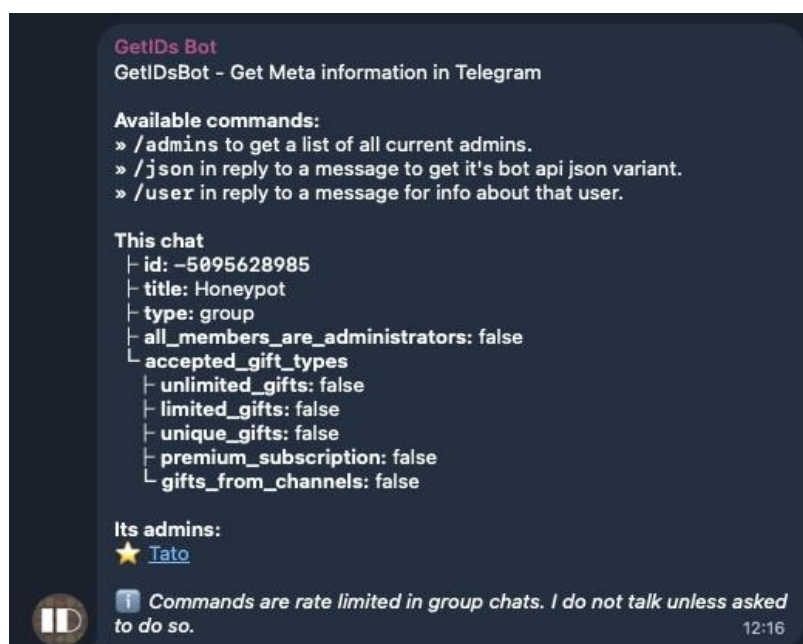
```

Intentos de conexión remota e introducción de contraseñas

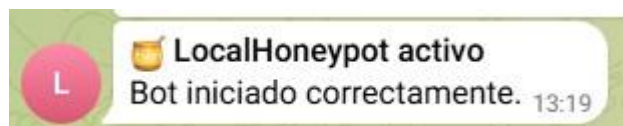
Kibana: ataque simulado de Izan Lozano al Honeypot

Anexo C — Capturas del sistema de alertas Telegram

Capturas del grupo de Telegram del equipo con los resúmenes de alertas recibidos durante el funcionamiento del honeypot.



Grupo de Telegram: mensaje de configuración del bot



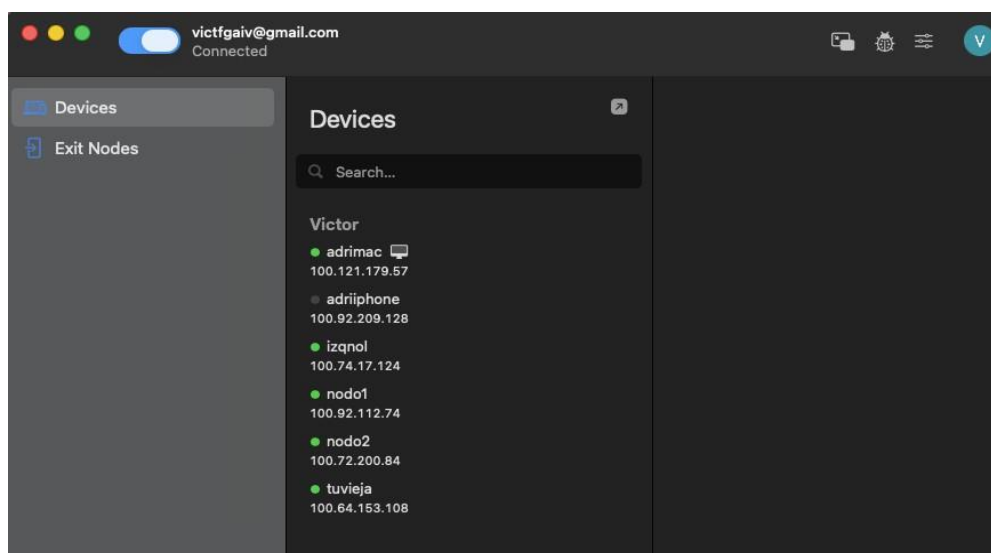
Grupo de Telegram: mensaje de inicio del bot



Grupo de Telegram: resumen de ataques con top países

Anexo D — Capturas de la infraestructura desplegada

Evidencias visuales de la infraestructura instalada y configurada.



Panel de Tailscale: dispositivos conectados a la red privada que tenemos

```
via@server:~$ docker ps | wc -l
40
```

```
via@server:~$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.RunningFor}}"
NAME      STATUS      CREATED
snare     Up 12 hours    12 hours ago
tanner     Up 12 hours    12 hours ago
kibana     Up 12 hours (healthy) 12 hours ago
logstash   Up 12 hours (healthy) 12 hours ago
map_data   Up 12 hours    12 hours ago
tanner_api Up 12 hours    12 hours ago
honeyaml   Up 12 hours    12 hours ago
redishoney Up 12 hours    12 hours ago
miniprint  Up 12 hours    12 hours ago
h0neytr4p  Up 12 hours    12 hours ago
ipphoney   Up 12 hours    12 hours ago
elasticpot Up 12 hours    12 hours ago
wordpot    Up 12 hours    12 hours ago
p0f        Up 12 hours    12 hours ago
medpot     Up 12 hours    12 hours ago
adbhoney   Up 12 hours    12 hours ago
sentrypeer Up 12 hours    12 hours ago
map_web     Up 12 hours    12 hours ago
nginx      Up 12 hours    12 hours ago
cowrie     Up 12 hours    12 hours ago
elasticsearch Up 12 hours (healthy) 12 hours ago
conpot_guardian_ast Up 12 hours (healthy) 12 hours ago
conpot_iec104 Up 12 hours (healthy) 12 hours ago
heralding   Up 12 hours    12 hours ago
mailoney    Up 12 hours    12 hours ago
ewsposter   Up 12 hours    12 hours ago
ciscoasa    Up 12 hours    12 hours ago
fatt        Up 9 hours     12 hours ago
spiderfoot  Up 12 hours (healthy) 12 hours ago
tanner_redis Up 12 hours    12 hours ago
tanner_phpox Up 12 hours    12 hours ago
suricata    Up 12 hours    12 hours ago
conpot_kamstrup_382 Up 12 hours (healthy) 12 hours ago
map_redis   Up 12 hours    12 hours ago
dionaea     Up 12 hours (healthy) 12 hours ago
conpot_ipmi Up 12 hours (healthy) 12 hours ago
dicompot    Up 12 hours    12 hours ago
honeytrap   Up 12 hours    12 hours ago
tpotinit    Up 12 hours (healthy) 12 hours ago
via@server:~$
```

Salida de Docker ps: 40 contenedores activos en el Nodo 1

```
via@server:~$ df -h
S.ficheros      Tamaño Usados  Disp Uso% Montado en
udev            5,8G      0    5,8G   0% /dev
tmpfs           1,2G    4,3M    1,2G   1% /run
/dev/sdb6       137G    45G    85G  35% /
tmpfs           5,9G      0    5,9G   0% /dev/shm
tmpfs           5,0M      0    5,0M   0% /run/lock
/dev/sda1       916G    28K   870G   1% /backup
/dev/sdb1       451M    81M   343M  19% /boot
/dev/sdb7       313G    28K   297G   1% /data
tmpfs           1,2G      0    1,2G   0% /run/user/0
tmpfs           1,2G      0    1,2G   0% /run/user/1000
```

```
via@server:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          11Gi         8,8Gi         183Mi        3,0Mi        2,7Gi        2,6Gi
Swap:         7,4Gi         620Mi         6,8Gi
```

Discos y Ram NODO 1 Verificación de discos y RAM

```
izqno1@debian:~$ df -h
S.ficheros      Tamaño Usados  Disp Uso% Montado en
udev            3,9G      0    3,9G   0% /dev
tmpfs           794M    3,5M   791M   1% /run
/dev/sde3       46G     10G    34G  24% /
tmpfs           3,9G      0    3,9G   0% /dev/shm
tmpfs           5,0M    4,0K    5,0M   1% /run/lock
/dev/sde1       462M    90M   344M  21% /boot
/dev/sde4       97G     24K    92G   1% /data
tmpfs           794M    4,0K   794M   1% /run/user/1000
izqno1@debian:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7,7Gi         4,3Gi         1,6Gi        3,0Mi        1,8Gi        3,2Gi
Swap:         4,7Gi           0B         4,7Gi
```

Discos y Ram NODO 2 Verificación de discos y RAM